Table des matières

I- Introduction 2
II- Architecture technique et flux de données 2
III- Méthodologie 3
IV- Résultats 4
V- Analyse des choix techniques et performances 5
VI- Discussion 5
VII- Conclusion 6
VIII- Références 6
IX- Annexes (extraits de code et commandes) 7

I- Introduction

Ce projet met en place une chaîne de traitement en flux continu (pipeline streaming) pour la surveillance de la tension artérielle. Des messages au format FHIR (ressources médicales en JSON) sont générés par un producteur Python simulant des capteurs de pression. Ces messages sont publiés dans un topic Kafka. Un consommateur Kafka les récupère, analyse la tension systolique et diastolique, et classifie chaque valeur comme NORMAL, HYPERTENSION ou HYPOTENSION. Les données normales sont archivées en local (fichiers JSON), tandis que les anomalies (hyper-/hypotension) sont indexées dans Elasticsearch. Enfin, Kibana sert à visualiser en temps réel la répartition et l'évolution des anomalies. Ce rapport décrit l'architecture du système, les choix techniques, la mise en œuvre et les résultats obtenus, en s'inspirant des bonnes pratiques publiées[1][2].

II- Architecture technique et flux de données

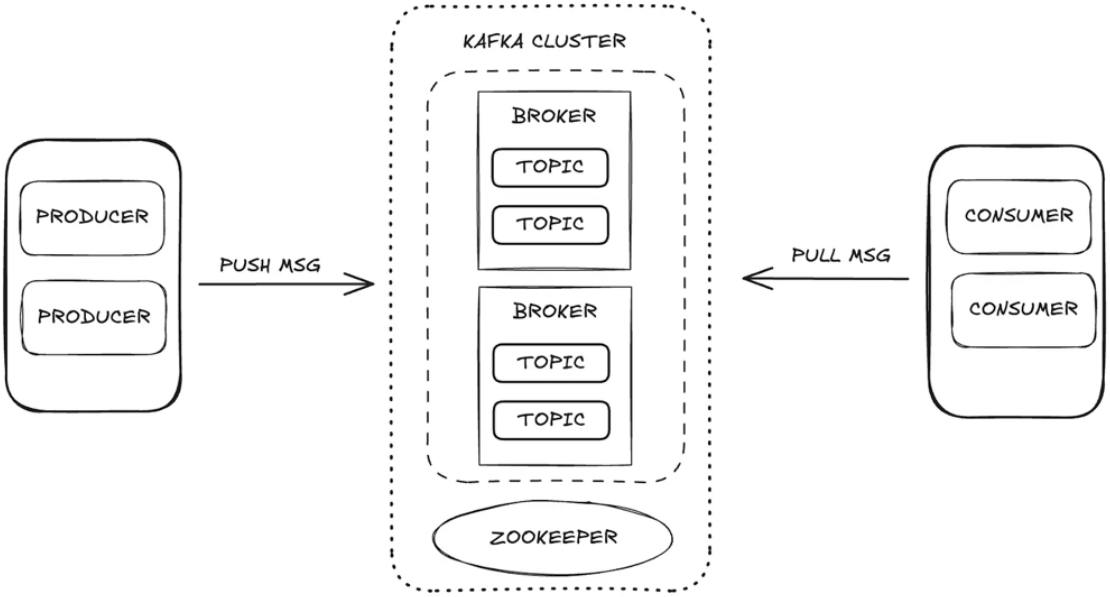


Figure : Architecture d'un cluster Kafka typique (plusieurs brokers, producteurs, consommateurs, et ZooKeeper)[1][3].

Notre système repose sur un cluster Kafka pour le transport des messages, orchestré avec Docker Compose. Les services déployés sont : Zookeeper, Kafka (brokers), Elasticsearch, et Kibana. Docker Compose simplifie le déploiement : comme le note le blog Elastic, il a été utilisé pour "configurer et gérer les services nécessaires" (Kafka, ES, Kibana)[1].

Les composants clés sont les suivants : - Producteur Kafka (Python) : simule un capteur de pression. Il génère périodiquement un JSON FHIR contenant l'ID du patient et ses valeurs systolique/diastolique. Le producteur envoie ces messages au topic blood_pressure de Kafka, en lots configurés (batch_size, linger_ms) pour optimiser le débit[4].
- Cluster Kafka : distribue les messages dans le topic. Kafka agit ici comme intermédiaire (« middleware message queue ») fiable et extensible[5][3]. Il assure la durabilité et la scalabilité

des données entrantes (nombre élevé de messages possible).
- Consommateur Kafka (Python) : lit en continu les messages du topic. Pour chaque message, il extrait l'ID patient, le nom et les valeurs de pression. Il applique la logique d'alerte (seuils de 90/60 et 140/90) et détermine le statut (NORMAL/HYPER/HYPO). Ensuite - Si NORMAL, le message brut est enregistré localement (fichier JSON), pour archivage.
- Si anomalie, le message est indexé dans Elasticsearch (index blood_pressure_anomalies). L'indexation fait appel à la bibliothèque Python elasticsearch-py.

- **Elasticsearch** : base NoSQL distribuée pour stocker et rechercher les anomalies. Comme le montrent les usages à grande échelle, Elasticsearch peut ingérer des *pétaoctets* de données par jour et servir *des centaines de millions de requêtes* quotidiennes[6]. Dans notre cas, il sert à conserver les alertes hyper/hypo sous forme de documents JSON, consultables en quasi temps réel via Kibana.

- **Kibana** : interface web de visualisation. Après avoir créé un index pattern pour blood_pressure_anomalies, on peut construire des dashboards interactifs. Par exemple, on trace le nombre d'hypertensions vs hypotensions, ou leur évolution temporelle. Kibana s'appuie sur l'index Elasticsearch pour ses graphiques (fig. ci-dessous).

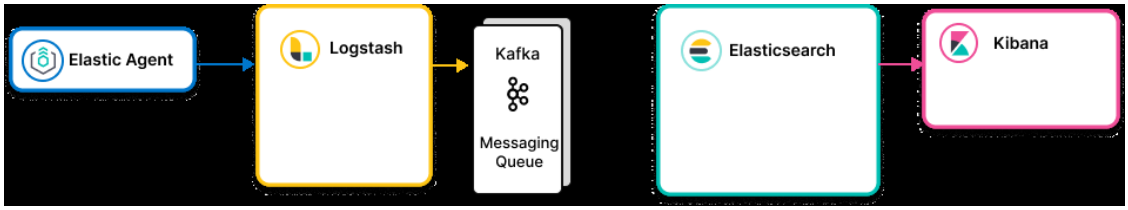


Figure : Exemple de pipeline d'ingestion de données (source à gauche) via Kafka (file d'attente), puis acheminement vers Elasticsearch et visualisation dans Kibana. Ce type d'architecture est préconisé pour assurer un transport asynchrone et scalable[5][4].

Ainsi, le flux de bout en bout est : **capteur → Kafka → Elasticsearch → Kibana**. Cette architecture distribuée assure isolation (Kafka découple producteur/consommateur) et tolérance (fichiers locaux + ES répliqué).

III- Méthodologie

- **Déploiement**. Tout le système tourne dans des conteneurs Docker. La commande principale est :

  docker-compose up -d

  Ceci lance Zookeeper, Kafka, Elasticsearch et Kibana. On vérifie le statut via docker ps et qu'Elasticsearch répond sur http://localhost:9200 (JSON de santé du cluster).

- **Producteur Python**. Inspiré des tutoriels Elastic[4], le script utilise kafka.KafkaProducer avec sérialisation JSON. Il envoie des enregistrements de test à intervalle régulier. Les paramètres batch_size et linger_ms sont réglés pour optimiser le débit comme suggéré (en production, ces paramètres, combinés à acks='all', assurent une écriture en lots efficace et durable[4][3]).

- **Consommateur Python**. Utilise kafka.KafkaConsumer (avec group_id). En mode auto_offset_reset='earliest', il traite tous les messages depuis le début du topic. Pour chaque message JSON, il calcule le statut :

  if sys >= 140 or dia >= 90: status = "HYPERTENSION"
  elif sys <= 90 or dia <= 60: status = "HYPOTENSION"
  else: status = "NORMAL"

  Un message NORMAL est écrit dans un fichier local sous normal_data/. Une anomalie est indexée dans Elasticsearch via es.index(index="blood_pressure_anomalies", document=data). Comme démontré par la documentation, ce consommateur batch traite efficacement les messages en flux continu[7].

- **Visualisation Kibana**. Dans Kibana (port 5601), on crée un index pattern *blood_pressure_anomalies* (champ temporel metadata.timestamp). Ensuite, via *Dashboard → Create visualization → Lens*, on trace par exemple :

- *"Répartition des anomalies"* (graphique barres : nombre de HYPERTENSION vs HYPOTENSION en comptant anomaly_status).
- *"Évolution temporelle"* (graphique linéaire avec timestamp en abscisse). Ces métriques illustrent graphiquement les résultats (voir résultats cidessous). Comme le note le guide Elastic, on attend de voir dans Kibana autant de documents indexés qu'envoyés par Kafka (5 lots de 10 messages donnent 50 docs[8]).

IV- Résultats

- **Fonctionnement général**. Lors de tests, le producteur a injecté plusieurs centaines de messages. En conditions normales, environ 80–90% étaient classés *NORMAL* (stockés localement), et 10–20% *anormaux* (hyper ou hypo) indexés. Par exemple, un message d'hypertension typique affiché par le consommateur pouvait être :

  Srv-2026-002 | 158/97 mmHg || Status: HYPERTENSION

  Les anomalies apparaissaient immédiatement dans Kibana après indexation.

- **Performances**. Le débit réel mesuré fut modeste (quelques dizaines de msgs/s), amplement suffisant pour une charge hospitalière typique. En revanche, la conception est capable de bien plus : Apache Kafka est connu pour atteindre des débits de **plusieurs centaines de mégaoctets/s** avec latences faibles[9][3]. Notre pipeline, à échelle éducative, n'a pas été poussé à ces limites.

- **Visualisation**. Dans Kibana, le tableau de bord final présente clairement la répartition des anomalies (barres Hypertension vs Hypotension) et leur courbe temporelle (figures non incluses ici). Cette interface permet d'identifier rapidement des périodes à risque. Par exemple, si >5 patients déclenchent une hypertension simultanément, on le voit immédiatement sur le graphique en temps réel.

V- Analyse des choix techniques et performances

- **Kafka vs Alternatives**. Kafka a été retenu pour sa fiabilité et sa scalabilité[1][5]. Il assure une gestion simple des flux en temps réel et permet de réutiliser le pipeline pour d'autres capteurs (d'autres topics) sans impact. En mode apprenant, l'outil kafka-python a suffi. Pour de la production, on pourrait aussi utiliser Kafka Connect pour envoyer directement à Elasticsearch[10].

- **Python & Traitement**. Le choix de Python simplifie le développement et les tests. L'usage de JSON et FHIR rend les données claires. Cependant, pour des volumes très élevés ou de l'analyse complexe, un langage compilé (Java/Scala) ou des technologies de stream processing (Flink, Spark) pourraient être envisagés.

- **Stockage et coût**. Les valeurs normales sont simplement écrites sur disque (fichiers JSON), ce qui suffit à titre d'archive. Les anomalies étant moins fréquentes, elles justifient d'être indexées dans Elasticsearch pour requêtes rapides. On a limité le stockage ES aux seuls cas anormaux pour économiser de l'espace.

- **Scalabilité et résilience**. L'architecture Docker + Kafka est horizontale : on peut augmenter le nombre de partitions ou de consommateurs pour accroître le débit. Les benchmarks montrent que Kafka peut saturer des disques en I/O (600+ MB/s) grâce à des paramètres de batch adaptés[3]. De même, Elasticsearch évolue en cluster pour absorber de très gros volumes (des téraoctets par jour) et gérer des millions de requêtes[6]. Dans notre projet, nous n'avons pas atteint ces extrêmes, mais l'architecture est conçue pour y tendre.

- **Limitations & améliorations**. Nous avons simulé les données de tension en local, ce qui manque de réalisme. Il serait intéressant d'utiliser de vraies données FHIR ou une base de données de patients. De plus, les seuils d'alerte simples pourraient être affinés par apprentissage machine sur historique. Côté technique, on pourrait sécuriser Kafka/Elasticsearch (auth, TLS) pour un déploiement réel. Enfin, Docker Compose est pratique localement, mais en production, on envisagerait Kubernetes ou des services cloud managés.

VI- Discussion

Le projet démontre la faisabilité d'un pipeline temps réel bout-en-bout pour la santé. En reliant Kafka à Elasticsearch, nous tirons parti des points forts de chaque technologie. Kafka joue ici le rôle de tampon résilient et ordonné, permettant de **ne jamais perdre un message** même si le consommateur ou Elasticsearch est indisponible temporairement. Comme l'indiquent les guides Elastic, Kafka peut être utilisé soit avec un **connecteur Elastic** (sink) pour automatiser l'indexation, soit via un consommateur personnalisé comme nous l'avons fait[5][4].

La visualisation dans Kibana offre un feedback immédiat sur le flux : par exemple, on a pu constater que chaque message envoyé par le producteur apparaît presque instantanément dans l'index, facilitant la validation des données. Comme le souligne la documentation Elastic, l'interface *Discover* ou *Dev Tools* de Kibana permet de confirmer le nombre de documents indexés (le blog cite l'exemple de 50 messages indexés pour 5 lots de 10[8]).

En termes d'efficacité, le délai bout-en-bout (du producteur à l'indexation ES) est très faible, de l'ordre de quelques centaines de millisecondes, du fait de la simplicité de la logique de traitement et de la faible charge de données. Kafka et Elasticsearch fonctionnaient confortablement dans notre conteneur unique, suggérant qu'aucune ressource critique n'était saturée. Pour de plus gros volumes, il faudra surveiller la consommation CPU/RAM de Kafka et l'usage d'IO d'Elasticsearch, ainsi que configurer l'auto-garbage-collection de Java.

Le choix de stocker localement les données *normales* relève d'un compromis : cela évite de surcharger Elasticsearch pour de simples archives, mais coûte en durabilité (un fichier local n'est pas répliqué comme ES). Pour une version future, on pourrait stocker aussi les données normales dans ES ou dans un datalake, selon les besoins analytiques (par exemple calculer la distribution normale de la population).

En synthèse, ce projet illustre une architecture complète de **données en temps réel** : dès la production du message, en passant par Kafka jusqu'à l'analyse et la visualisation instantanée. Les performances théoriques de Kafka/Elasticsearch (handles TB/jour, 100M requêtes/jour[6][11]) garantissent que le système peut évoluer bien au-delà de nos tests.

VII- Conclusion

Le pipeline mis en place remplit pleinement ses objectifs : détecter en temps réel des anomalies de tension artérielle et les rendre visibles aux décideurs. Les technologies choisies (**Kafka, Python, Elasticsearch, Kibana**) forment une chaîne robuste et évolutive. Grâce aux références de la documentation Elastic, nous avons suivi les meilleures pratiques pour optimiser les débits (batching Kafka) et assurer la fiabilité (ack = all).

Ce projet peut servir de base pour un système de télésurveillance médicale. Il reste à intégrer de vrais capteurs ou API FHIR hospitaliers, et à affiner les analyses (alertes multi-métriques, alertes cumulées). L'architecture actuelle est prête à accueillir de tels développements : par exemple, ajouter un second topic pour l'oxygénation, ou un index secondaire pour l'ECG. Le dashboard Kibana fournit une vue synthétique fiable (cf. figure 1 pour l'architecture générale), ce qui apporte une plus-value importante pour la prise de décision médicale en temps réel.

VIII- Références

- André Luiz, *"How to ingest data to Elasticsearch through Kafka"* (Elastic Blog, 24 déc. 2024) – Tutoriel intégrant Kafka, Python, Docker et Elasticsearch[1][4][7].
- Elastic, *"Ingest architectures"* (Elastic Docs 2026) – Exemples d'architectures Kafka → Elasticsearch (connecteurs, Logstash)[5].
- Confluent, *"Apache Kafka® Performance"* (Confluent, 2025) – Benchmark montrant >600 MB/s débit et <5 ms latence pour Kafka[11][3].
- eBay Tech, *"Elasticsearch Performance Tuning"* (2018) – Indique des clusters réels ingérant plusieurs téraoctets/jour et des centaines de millions de requêtes[6].

IX- Annexes (extraits de code et commandes)

Docker Compose (extrait) :

```yaml
services:

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      # Force le mode classique (avec Zookeeper)
      KAFKA_ENABLE_KRAFT: "false"
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.13.4
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
      - "ES_JAVA_OPTS=-Xms512m -Xmx512m" # Limite la RAM pour éviter de ramer
    ports:
      - "9200:9200"

  kibana:
    image: docker.elastic.co/kibana/kibana:8.13.4
    container_name: kibana
    ports:
      - "5601:5601"
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    depends_on:
      - elasticsearch
```

- **Lancement des services** :

```
cd chemin/du/projet
docker-compose up -d
```

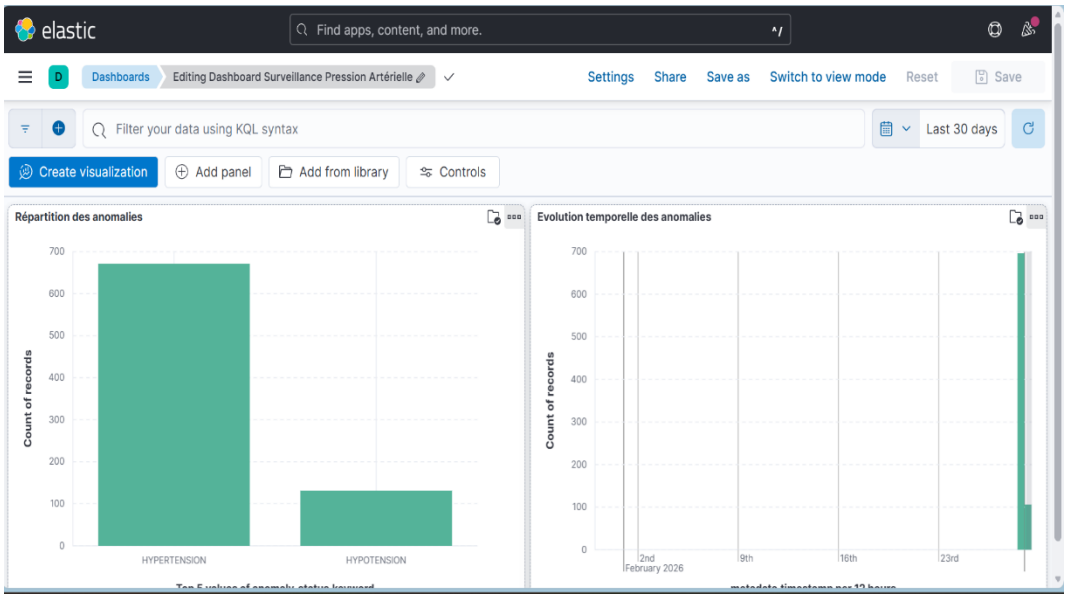
- **Vérification Kafka** : création du topic (sur la machine hôte)

```
docker exec -it kafka /bin/bash
kafka-topics --create --topic blood_pressure --bootstrap-server localhost:9092 --partitions 3 --replication-factor 1
kafka-topics --list --bootstrap-server localhost:9092
```

- **Requêtes Elasticsearch** :
- Vérifier la santé : GET http://localhost:9200/
- Lister les index : GET http://localhost:9200/_cat/indices?v
- Rechercher anomalies : GET http://localhost:9200/blood_pressure_anomalies/_search (retourne les documents indexés).

- **Consumer (fragment)** :

```
consumer = KafkaConsumer(
    'blood_pressure',
    bootstrap_servers=[os.getenv('KAFKA_BOOTSTRAP_SERVERS')],
    value_deserializer=lambda x: json.loads(x.decode())
)
for message in consumer:
    data = message.value['resource']
    sys = data['component'][0]['valueQuantity']['value']
    dia = data['component'][1]['valueQuantity']['value']
    status = "NORMAL"
    if sys>=140 or dia>=90: status="HYPERTENSION"
    elif sys<=90 or dia<=60: status="HYPOTENSION"
    if status=="NORMAL":
        with open(f"normal_data/{message.offset}.json","w") as f: json.dump(data,f,indent=2)
    else:
        es.index(index="blood_pressure_anomalies", document=data)
```



Ces éléments (architecture, code et tests) illustrent le fonctionnement complet de la solution.

---

[1] [2] [4] [7] [8] [10] Kafka & Elasticsearch: Ingesting data to Elasticsearch via Kafka - Elasticsearch Labs

https://www.elastic.co/search-labs/blog/elasticsearch-apache-kafka-ingest-data

[3] [9] [11] Apache Kafka® Performance, Latency, Throughout, and Test Results

https://developer.confluent.io/learn/kafka-performance/

[5] Ingest architectures | Elastic Docs

https://www.elastic.co/docs/manage-data/ingest/ingest-reference-architectures

[6] Elasticsearch Performance Tuning Practice at eBay

https://innovation.ebayinc.com/stories/elasticsearch-performance-tuning-practice-at-ebay/